# Crunch Convos: **Sets & Dictionaries**

**Attendance**

linktr.ee/CODE.CRUNCH

# Hashing

Hashing is a technique that maps data to a fixed-size value called a **hash value**.

**Hash Function:** Takes an input (like a key) and converts it into a fixed-size number.

- **Example:** Hashing the string "apple" might return the integer 123456.

**Hash table:** a data structure that stores key-value pairs and uses hashing to index the keys.

- A hash function is applied to the key to generate a hash value.
- This hash value determines where the key-value pair will be stored in the table (i.e., an index or "bucket").

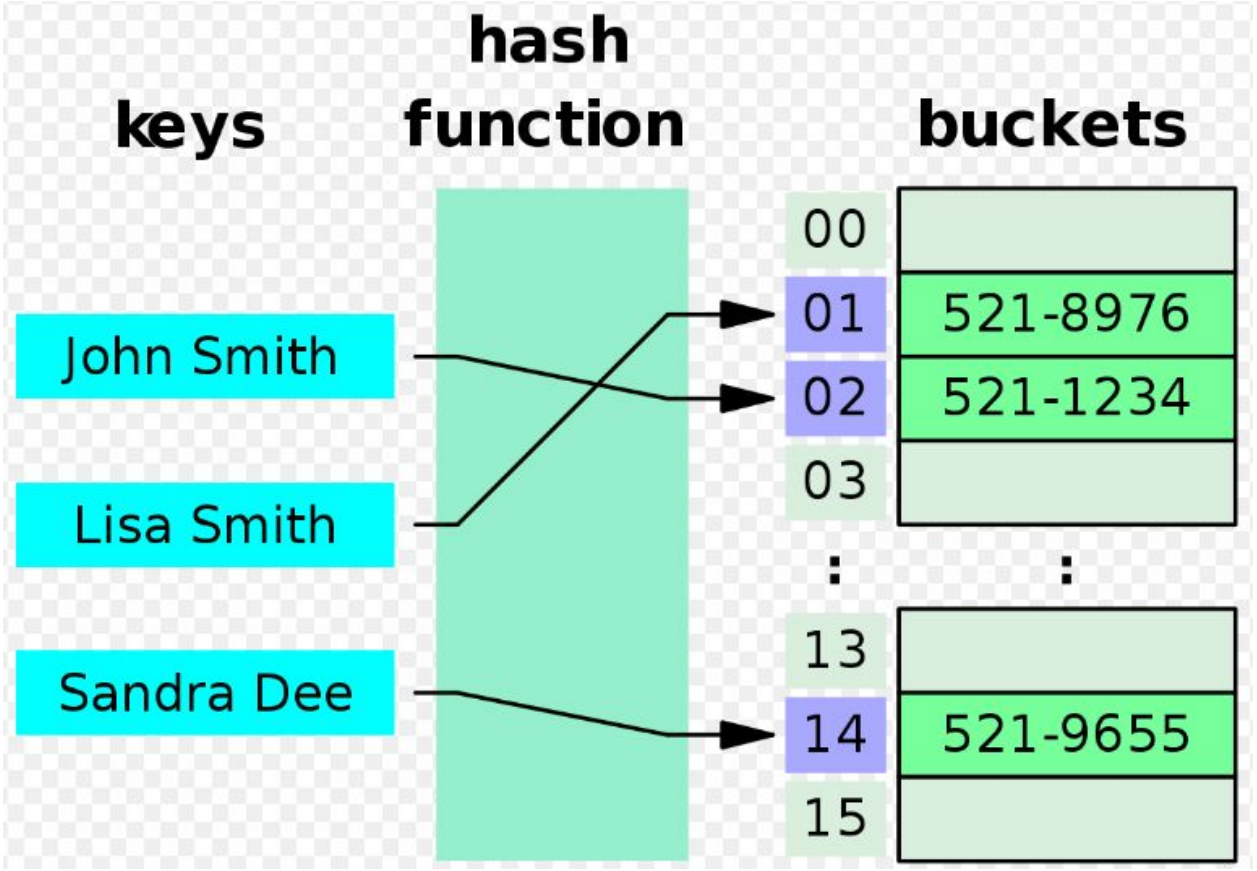


Fig. : Benjaminson, Emma . *"Hash Tables"* , 16 Jan. 2021, https://sassafras13.github.io/HashTables/.

# Hash Tables

**Fast operations:** Provide O(1) time complexity for:

- Insertion
- Deletion
- Lookup

**Used in sets:** Python sets use hash tables internally to ensure unique elements and fast membership checks.

**Used in dictionaries:** Python dictionaries are implemented using hash tables, allowing for efficient key-value pair lookups.

**Handles collisions:** When two keys map to the same index (collision), hash tables use techniques like chaining or open addressing to resolve them.

# Sets

A set is an unordered collection of unique items.

**Key Characteristics:**

- No duplicates.
- Unordered (no indexing or slicing).
- Mutable

```
# Creating a set
my_set = {1, 2, 3, 4}

# Empty Set
empty_set = set()   # NOT {}


# Get size of set
n = len(my_set) # n = 4
```

# Add an item

```
my_set = {1, 2, 3, 4}

my_set.add(5) # my_set = {1, 2, 3, 4, 5}

my_set.add(2)  # my_set = {1, 2, 3, 4}
```

# Remove item

```
my_set = {1, 2, 3, 4}

my_set.remove(2) # my_set = {1, 3, 4}

my_set.discard(2) # my_set = {1, 3, 4}

my_set.discard(7) # Raises KeyError since 7 is not present

my_set.clear() # my_set = set()
```

# Lookup an item

```
fruits = {'apple', 'banana', 'cherry', 'orange', 'pineapple'}

if 'cherry' in fruits:
    print('cherry!')

if 'tomato' not in fruits:
    print('not a fruit!')
```

# Loop through a Set

```
fruits = {'apple', 'banana', 'cherry', 'orange', 'pineapple'}

for fruit in fruits:
    print(fruit)
```

# Set Operations

```
A = {1, 2, 3}
B = {3, 4, 5}

union = A.union(B) # {1, 2, 3, 4, 5}
intersection = A.intersection(B) # {3}
difference = A.difference(B) # {1, 2, 4, 5}
```

# Dictionaries

A dictionary is a collection of key-value pairs.

**Key Characteristics:**

- Keys must be unique.
- Values can be any data type.
- Dictionaries are mutable.

```
# Creating dictionary
my_dict = {
  "name": "Alice",
  "age": 30,
  "city": "New York"
}

# Empty dictionary
my_dict = {}

# Get size of dictionary
n = len(my_dict) # n = 3
```

# Lookup a key

```
my_dict = {
   "name": "Alice",
   "age": 30,
   "city": "New York"
}

if "name" in my dict:
    print('I have a name')
```

# Look up a value using a key


```
scores = {
  "Brian": 78,
  "John": 85,
  "Michael": 92
}

brian_score = scores['Brian'] # brian_score = 78

alex_score = scores[‘Alex'] # Raises KeyError since 'Alex' is not a key
```

# Add/Remove Key-Value Pairs


```
my_dict = {
  "name": "Alice",
  "age": 30,
  "city": "New York"
}

my_dict["email"] = "alice@gmail.com"  # Adds a new key-value pair

my_dict["age"] = 31  # Updates existing value

del my_dict["city"] # Removes ("city": "New York") key-value pair

my_dict.clear() # my_dict = {}
```

# Common Dictionary Methods

- `.get()`: Safe access to values without error if key doesn't exist.
- `.keys()`: Returns all keys.
- `.values()`: Returns all values.
- `.items()`: Returns key-value pairs as tuples.

```
scores = {
    "Brian": 78,
    "John": 85,
    "Michael": 92  }

print(scores.get("John"))
# Output: 85

print(scores.get("Alice"))
# Output: None (Default if 'Alice' doesn't
exist)

print(scores.keys())
# Output: ['Brian', 'John', 'Michael']

print(scores.values())
# Output: [78, 85, 92]

print(scores.items())
# Output: [('Brian', 78), ('John', 85),
('Michael', 92)]
```

# Looping through Keys

```
for key in my_dict:
    print(key)

'''
name
age
city
email
'''
```

# Looping through Values

```
for value in my_dict.values():
    print(value)

'''
Alice
31
New York
alice@gmail.com
'''
```

# Looping through Key-Value Pairs

```
for key, value in my_dict.items():
    print(f"{key}: {value}")

'''
name: Alice
age: 31
city: New York
email: alice@gmail.com
'''
```

# Best Practices for Dictionaries and Sets

**When to Use Dictionaries:**

- **Mapping relationships:** Use dictionaries to associate keys with values (e.g., a name to a score, or a product to a price).
- **Quick lookups:** Dictionaries provide constant-time access to values by keys, making them ideal for fast lookups.
- **Common use in interviews:** Often used for counting occurrences or representing graphs.

**When to Use Sets:**

- **Unique collections:** Sets ensure that no duplicates are stored, making them perfect for tasks where uniqueness matters (e.g., removing duplicates from a list).
- **Efficient membership testing:** Sets provide constant-time checks to see if an element exists.
- **Common use in interviews :** Often used for problems involving intersection, union, and difference of collections, or finding unique values in a dataset.

# Problem 1 (Leetcode #217)

Given an integer array `nums`, return `True` if any value appears more than once in the array, otherwise return `False`

**Example 1:**
**Input:** nums = [1, 2, 3, 3]
**Output:** True

**Example 2:**
**Input:** nums = [1, 2, 3, 4]
**Output:** False

# Problem 2 (Leetcode #1)

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`

You may assume that each input would have exactly one solution, and you may not use the same element twice.

**Example 1:**

**Input:** nums = [2,7,11,15], target = 9

**Output:** [0,1]

**Example 2:**

**Input:** nums = [3,2,4], target = 6

**Output:** [1,2]

# Attendance

linktr.ee/CODE.CRUNCH

Join us for the next sessions

**Crunch Convos:** WED 2PM - 3PM
**Crunch Code:** FRI 1PM - 2PM

RSVP lu.ma/CODECRUNCH

Your feedback helps us improve. Share your thoughts!
**Go to our website:** go.fiu.edu/codecrunch